UserPage.Vue

- First time running coverage test (using v8)

% Coverage report from v8

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	51.41	96.03	53.33	51.41	
FlightRescheduleUpgrade.vue	100	100	100	100	
Flights.vue	100	100	100	100	
FlightsCancellation.vue	100	100	100	100	
IndexPage.vue	67.42	100	92.3	67.42	1-66
Login.vue	55.27	100	85.71	55.27	1-40,98-155
OrderHistory.vue	100	100	100	100	
UserPage.vue	24.49	88.88	10.71	24.49	...,335-336,345,349,353-358,362-366,369,373-446

- After adding scripts (using Istanbul)

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	60.86	67.84	65.27	62.31	
src	0	100	100	0	
App.vue	0	100	100	0	6-10
main.js	0	100	100	0	10-17
src/components/icons	0	0	0	0	
IconCommunity.vue	0	0	0	0	
IconDocumentation.vue	0	0	0	0	
IconEcosystem.vue	0	0	0	0	
IconSupport.vue	0	0	0	0	
IconTooling.vue	0	0	0	0	
src/router	100	100	100	100	
index.js	100	100	100	100	
src/stores	0	100	0	0	
counter.js	0	100	0	0	4-11
src/utils	62.5	66.66	100	62.5	
adjustAspectRatio.js	62.5	66.66	100	62.5	10-12
src/views	63.08	68.8	67.64	64.64	
FlightRescheduleUpgrade.vue	50	100	100	50	2
Flights.vue	100	96.29	100	100	133
FlightsCancellation.vue	50	100	100	50	2
IndexPage.vue	96.55	100	92.3	96.55	1
Login.vue	59.32	69.23	75	60.34	10,98-154
OrderHistory.vue	50	100	100	50	2
Register.vue	5.94	0	0	6.66	1,11,198-395,412-439
UserPage.vue	96.11	100	100	100	

Analysis

Reorganization with beforeEach **Hook**
Purpose: To avoid repetitive code and ensure a consistent test environment.
Implementation: Introduced a beforeEach block that mounts the UserPage component before each test. Configured global components and mocks within the beforeEach to be reused across all tests.

Mocking axios
Purpose: To simulate API calls without making actual HTTP requests, enabling controlled testing of asynchronous operations.
Implementation: Mocked the axios library using vi.mock('axios'). Used axios.post.mockResolvedValue({ data: {} }) to simulate successful API responses.

Expanded Test Cases
Purpose: To thoroughly test all functionalities of the UserPage component, including rendering, user interactions, validations, and method executions.
Implementation:

- **Navigation Bar Tests:**
  Verified that the navigation bar renders correctly.
  Tested redirection to the home page when the 'Home Page' button is clicked.
- **Sidebar Tests:**
  Ensured that the sidebar renders with the correct navigation links.
  Tested click events on sidebar navigation links.
- **Form Input Tests:**
  Checked that all form inputs render properly.
  Tested data binding by verifying that data properties update when inputs change.
- **Dropdown Tests:**
  Tested toggling of credit and debit card dropdowns.
  Verified that appropriate messages display when no cards are added.
- **Add Card Form Tests:**
  Tested rendering and interaction with the 'Add New Method' button.
  Verified that the add card form appears and that new card data updates correctly.
- **Validation Tests:**
  Added comprehensive tests for form validations, covering both valid and invalid inputs.
  Tested error messages for invalid first name, last name, email, postal code, date of birth, cardholder name, card number, expiry date, and CVV.
- **Method Testing:**
  Tested the saveChanges method for both successful data saving and failure due to validation errors.
  Tested the saveCard method for adding credit and debit cards, handling invalid card types, and resetting form fields after saving.
  Tested the getCardLogo method with various card numbers to ensure correct logo retrieval.
- **Asynchronous Behavior**:
  Used flushPromises to handle asynchronous operations in tests involving API calls and validation.
- **Edge Case Testing**:
  Ensured that optional fields can be empty and that the application handles missing required fields appropriately.
  Tested resetting of error messages upon validation.

Enhanced Component Interaction Testing
Purpose: To simulate real user interactions with the component and verify the application's response.
Implementation: Used Vue Test Utils methods like trigger, setValue, and setChecked to simulate clicks, input changes, and radio button selections. Verified the state of the component after interactions, such as the visibility of dropdowns and forms.

Improved Validation Testing
Purpose: To ensure that all validation logic works correctly and that users receive appropriate feedback.
Implementation: Tested invalid inputs for each form field and verified that the correct error messages display. Tested that error messages clear when inputs are corrected.

Testing Asynchronous Behavior
Purpose: To accurately test methods involving asynchronous operations, such as API calls and validation logic.
Implementation: Used flushPromises to wait for all pending promises to resolve before making assertions.